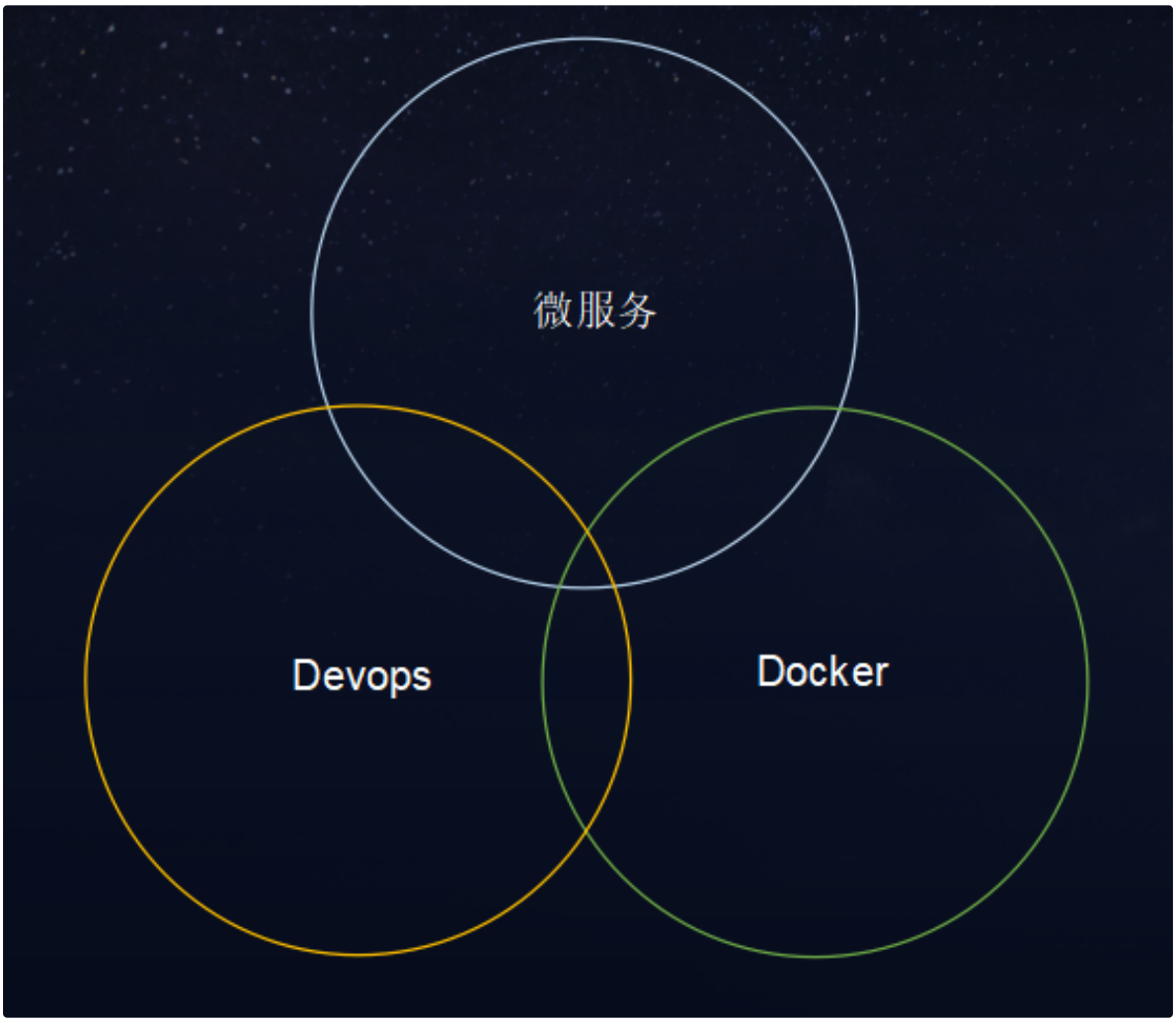
SRE-GitOps设计

通过GitOps的设计与落地，来满足云原生环境下业务的持续交付，建立标准的开发、 测试、生产环境， 形成统一的规范，完善自动化，为业务快速交付赋能。



整体架构

（蓝色代表后续做ARM兼容适配的地方）

请至钉钉文档查看「流程图」

基础环境

- IDC机房
  - AMD服务器
  - ARM服务器 **（资源采购&预算）**
  - 存储服务
- 域名
- 证书

Kubernetes

运行环境支持AMD和ARMv8

部署架构图

原则：**集群隔离**。一个环境一个集群，互不干扰。

目前已有四套集群，提供给研发、测试、生产以及Hotfix、poc使用。

预估信创适配需3-4套ARM集群环境。

后续共需管理7-8个集群环境

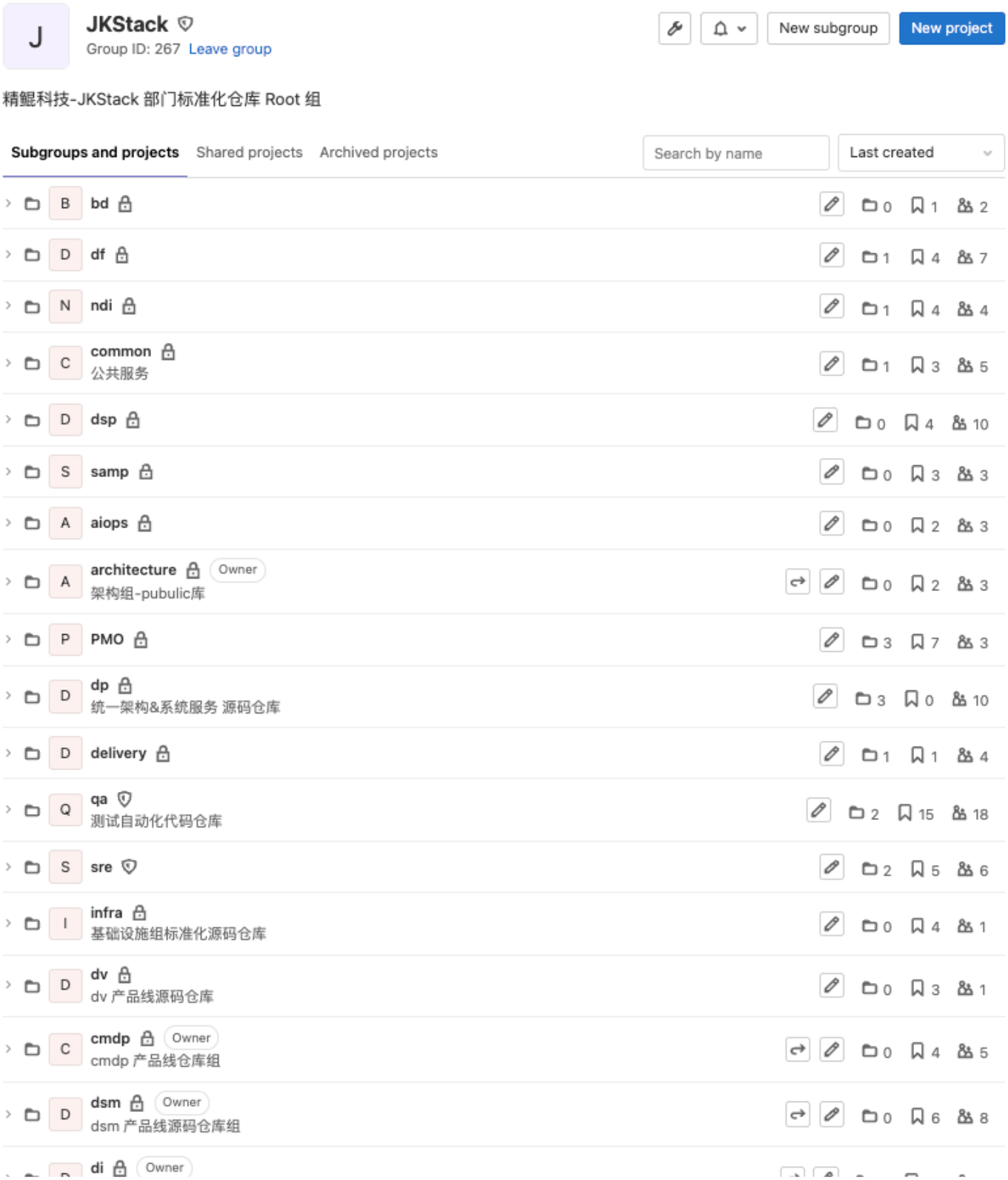
请至钉钉文档查看「流程图」

运维管理

代码仓库

通过流程管理，进行仓库的开通或变更，以产品组进行划分和管理，建立起唯一授信源，为后续Pipeline设计做好基础建设

JKStack代码仓库

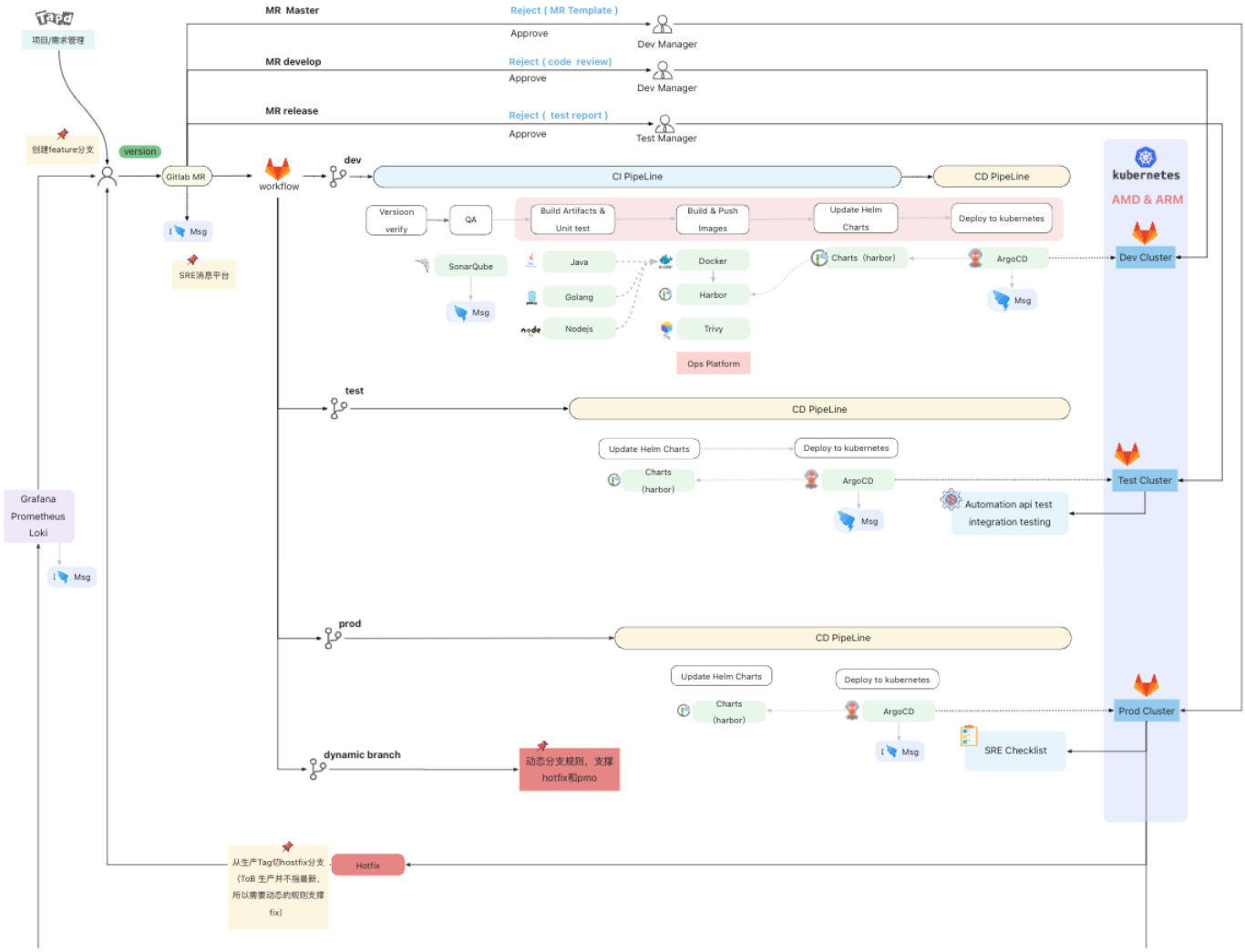


GitOps

原则： **采用**一切皆代码的原则，进行基础服务和业务项目的维护管理。**

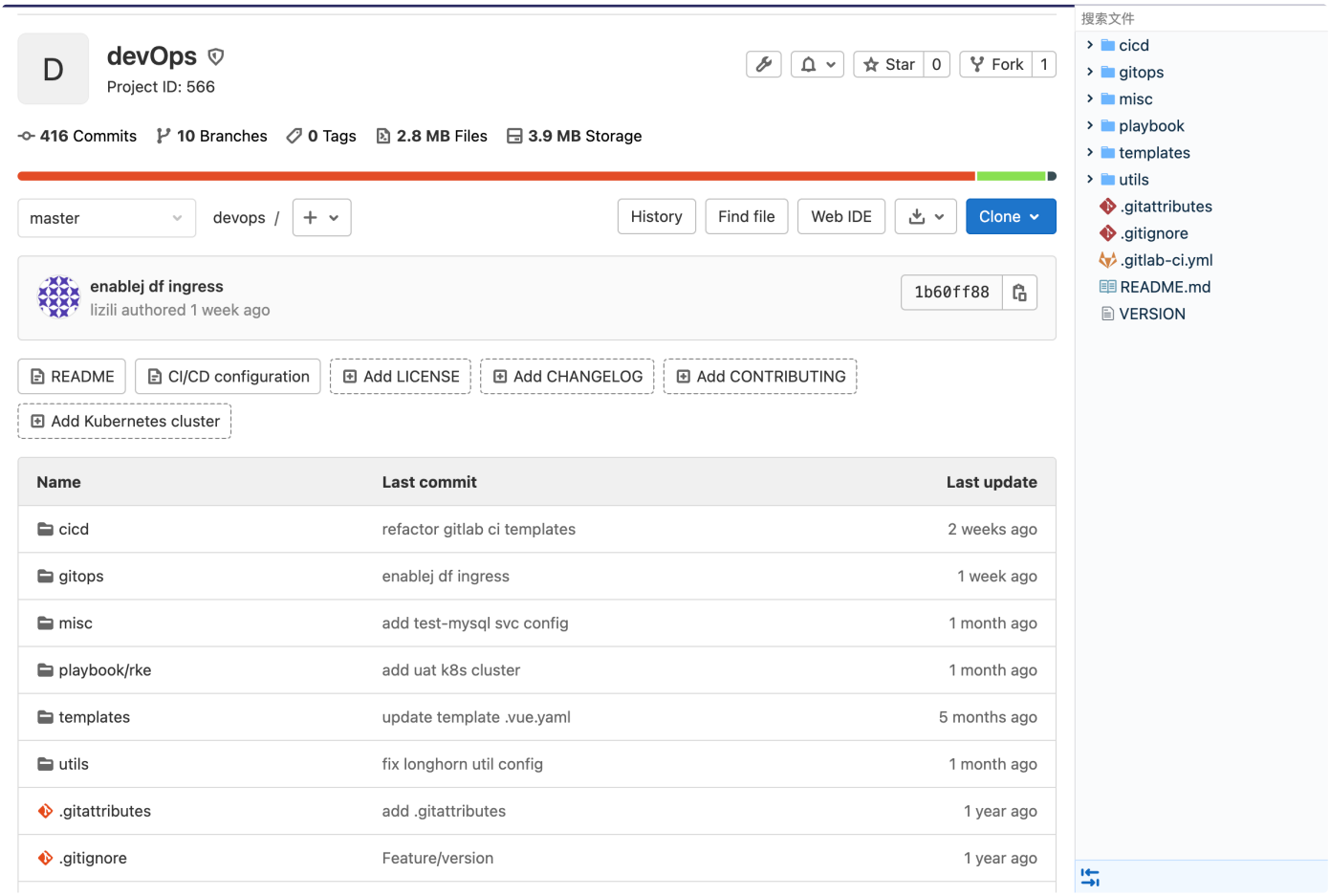
流程设计

从代码提交到项目上线全流程



仓库规划

运维工程项目规划**



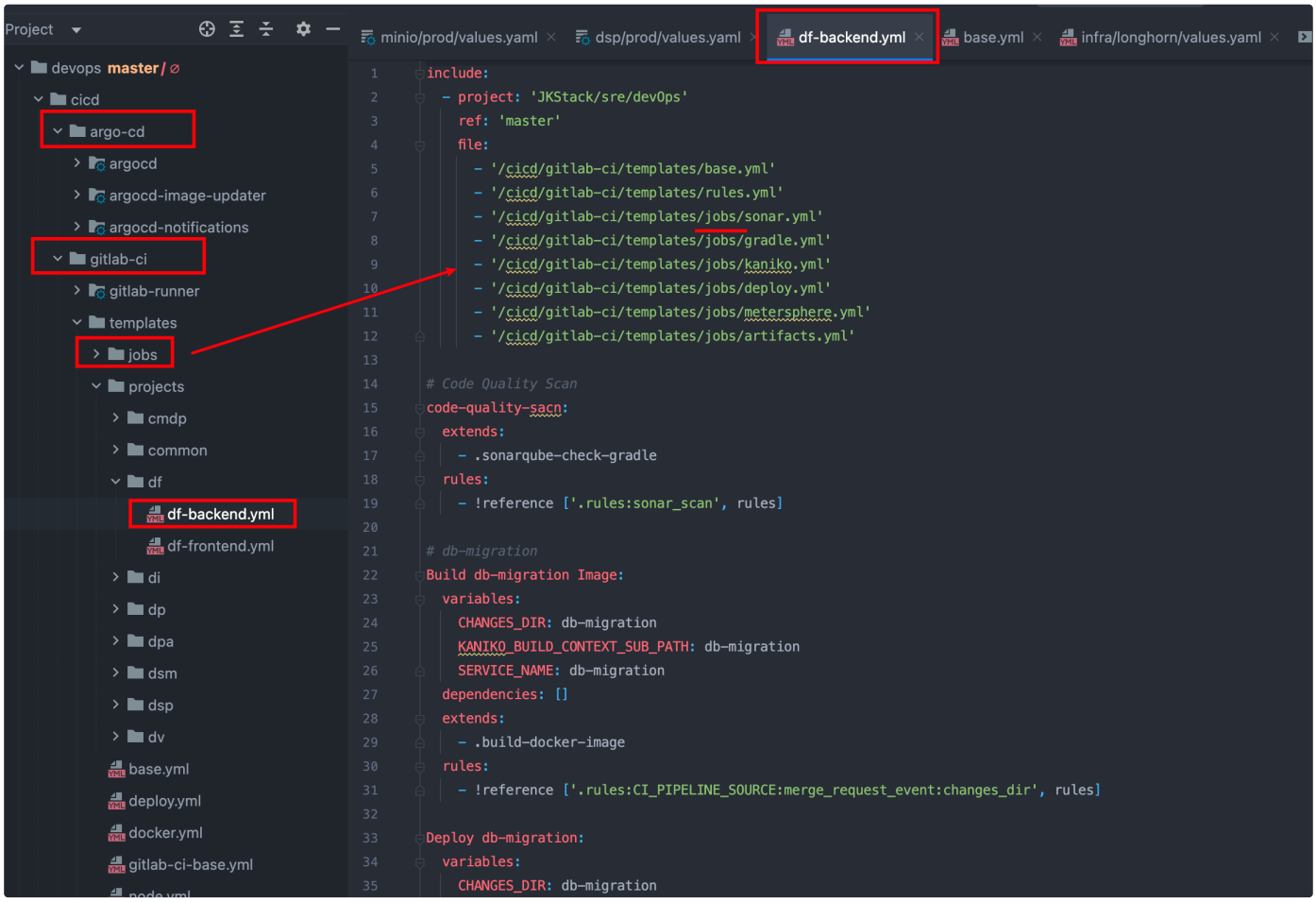
目录说明：

1. **cicd** ： 用来做产品的的打包和部署管理，产线内gitlab-ci直接引用即可，做到研发和运维代码管理上的解耦

1. **gitlab-ci.yaml**文件
    1. gitlab规定的固定命名文件，用来触发git pipeline，其内容编写遵循gitlab的编排语法，作用类似jenkinsfile
2. **gitops**： 用来做运维管理，如基础设施、中间件等
3. **misc**：试验性的临时功能。未集成到标准编排中的尝试性功能，均在此处编排和管理
4. **playbook**：Ansible剧本编排服务，目前提供基于rke工具拉起Kubernetes集群功能**(未支持arm)**
5. **template**: git pipeline 通用模板文件，现已弃用，迁移至cicd中
6. **utils**： 运维工具、程序管理，如mysql备份任务，kubernetes调试工具等

目录详解

cicd



1. gitlab-ci：项目的持续集成管理 **（ARM环境的制品产出，主要是在这里作适配）**
    1. gitlab-runner：ci任务的执行者，负责 ci 编排文件的处理和脚本的执行
    2. template： CI编排文件
        1. job： 提供通用基础服务，如gradle、maven、golang 、artifacts等pipeline节点服务
        2. projects： 按产线进行归类，按服务进行单独的编排管理，针对不同的服务，可在此进行模板的引用和特性的扩展

ci

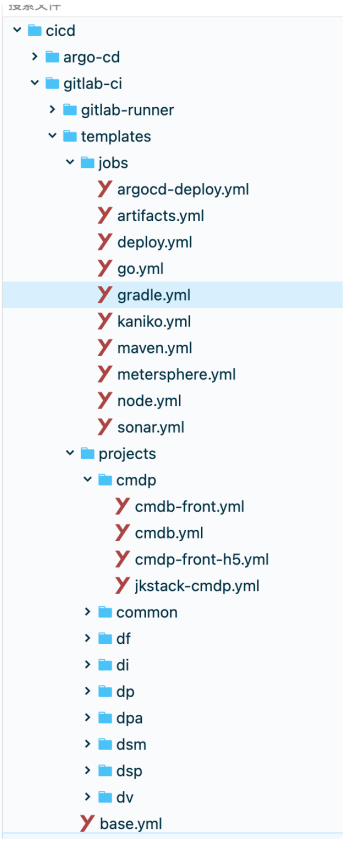
制品构建的所有操作均在此处调整，如和产线协商的变量新增(**XC_TONG_WEB_ENABLED**)以区分**东方通** jar依赖等，通过逻辑判断和处理，产出不同的制品。

gradle.yml 1.94 KB

Edit | Web IDE | Replace | Delete

```
.gradle-assemble:
  stage: package
  variables:
    GRADLE_OPTS: "-Dorg.gradle.daemon=false"
  image: ${REGISTRY_HOST}/cicd/gradle:${PACKAGE_GRADLE_DOCKER_VERSION} # 6.5.1-jdk8
  before_script:
    - |
      if [[ -d "${PROJECT_SUB_PATH:-}" ]]; then
        cd "${PROJECT_SUB_PATH}"
      fi
    - |
      export GRADLE_USER_HOME=`pwd`/.gradle
      if [[ -z "${MODULE-}" ]]; then
        echo "MODULE variable is not specfic, will assemble entire project."
      else
        export MODULE=$(echo "$MODULE" | tr / : | sed 's/^/:/' | sed 's/$/:/')
        echo "$MODULE will be assemble using gradle."
      fi
  script:
    - "gradle --build-cache ${MODULE}build -x test"
    - |
      export ZIPPED_ARCHIVE_API="$CI_API_V4_URL/projects/$CI_PROJECT_ID/jobs/$CI_JOB_ID/artifacts"
      echo "ZIPPED_ARCHIVE_API: ${ZIPPED_ARCHIVE_API}"
      echo "ZIPPED_ARCHIVE_API=${ZIPPED_ARCHIVE_API}" > artifacts-$CI_JOB_ID.env
  cache:
    key: "$CI_JOB_NAME"
    paths:
      - .gradle
  artifacts:
    when: on_success
    expire_in: 2 days
    paths:
      - $CI_PROJECT_DIR/**/build/libs/*.jar
    reports:
      dotenv: $CI_PROJECT_DIR/**/artifacts-$CI_JOB_ID.env
```

新增 XC_TONG_WEB_ENABLED



如何产出多环境下的制品?

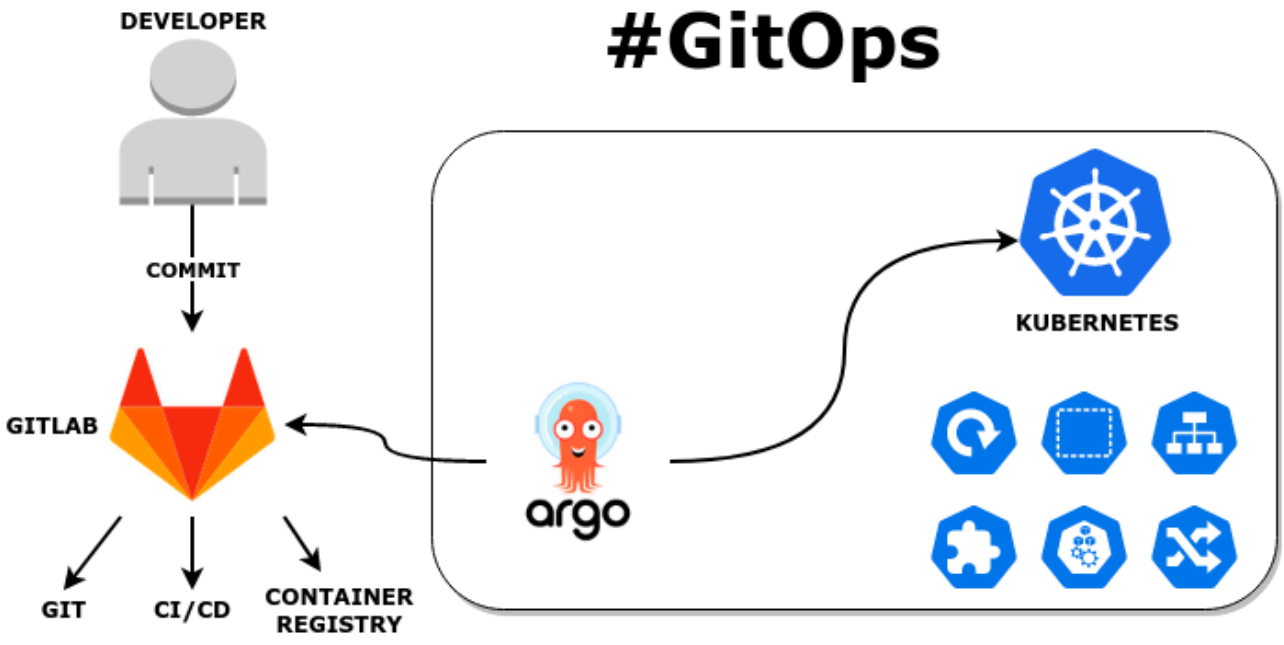
需调度runner到不同架构下的集群节点，进行编排文件的执行，这块逻辑会在gitlab-runner中进行兼容处理，并在 jobs 中做逻辑处理

cd

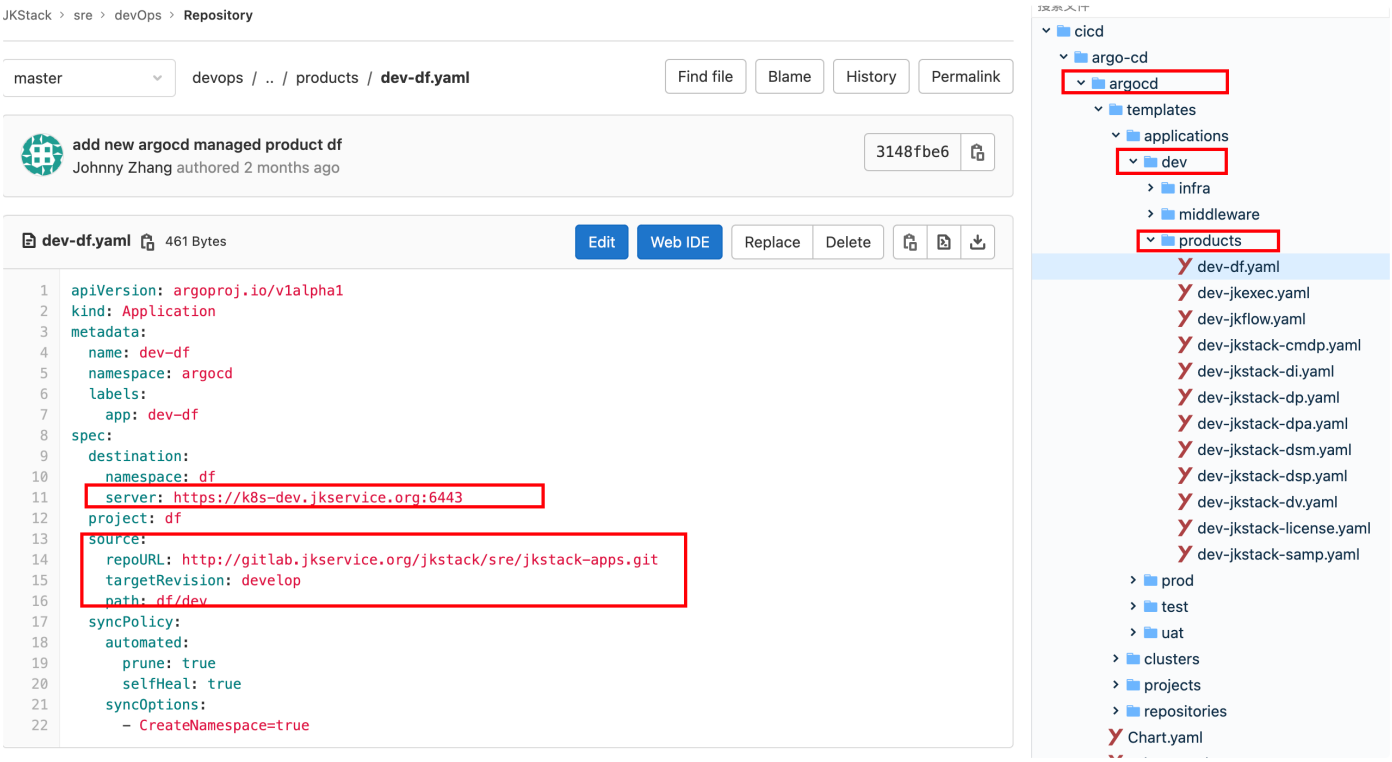
如何发布到不同的环境实行多环境的?

目前的发布是通过ArgoCD进行操作

- argocd是基于kubernetes的声明式的CD工具

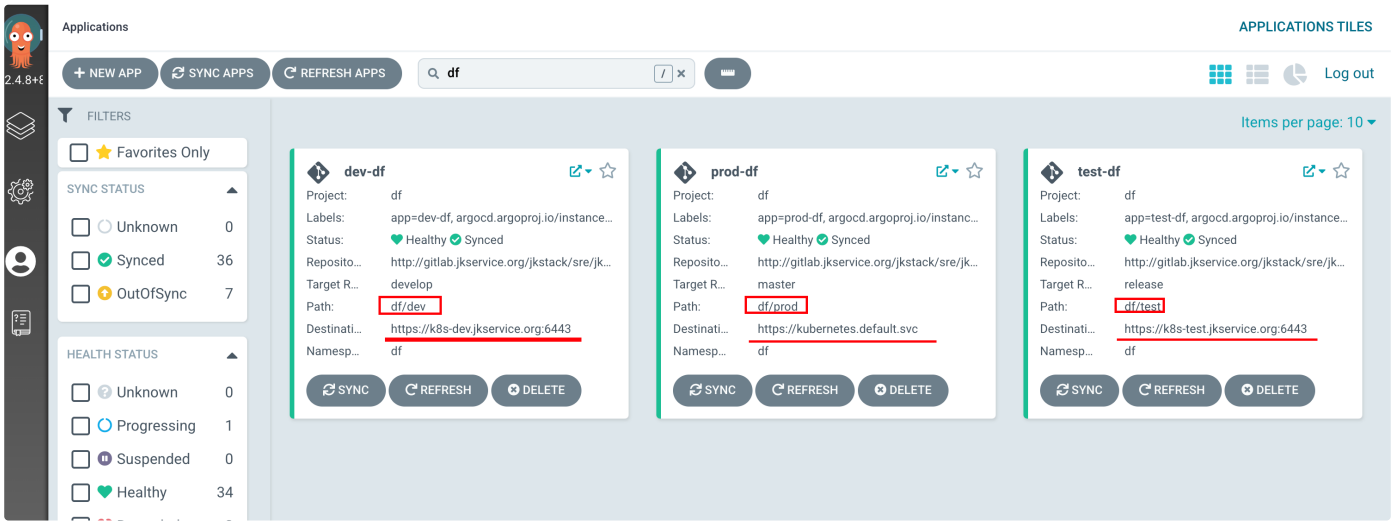


通过argo应用的创建和声明，来监控编排文件的变化，从而实现产品的CD操作

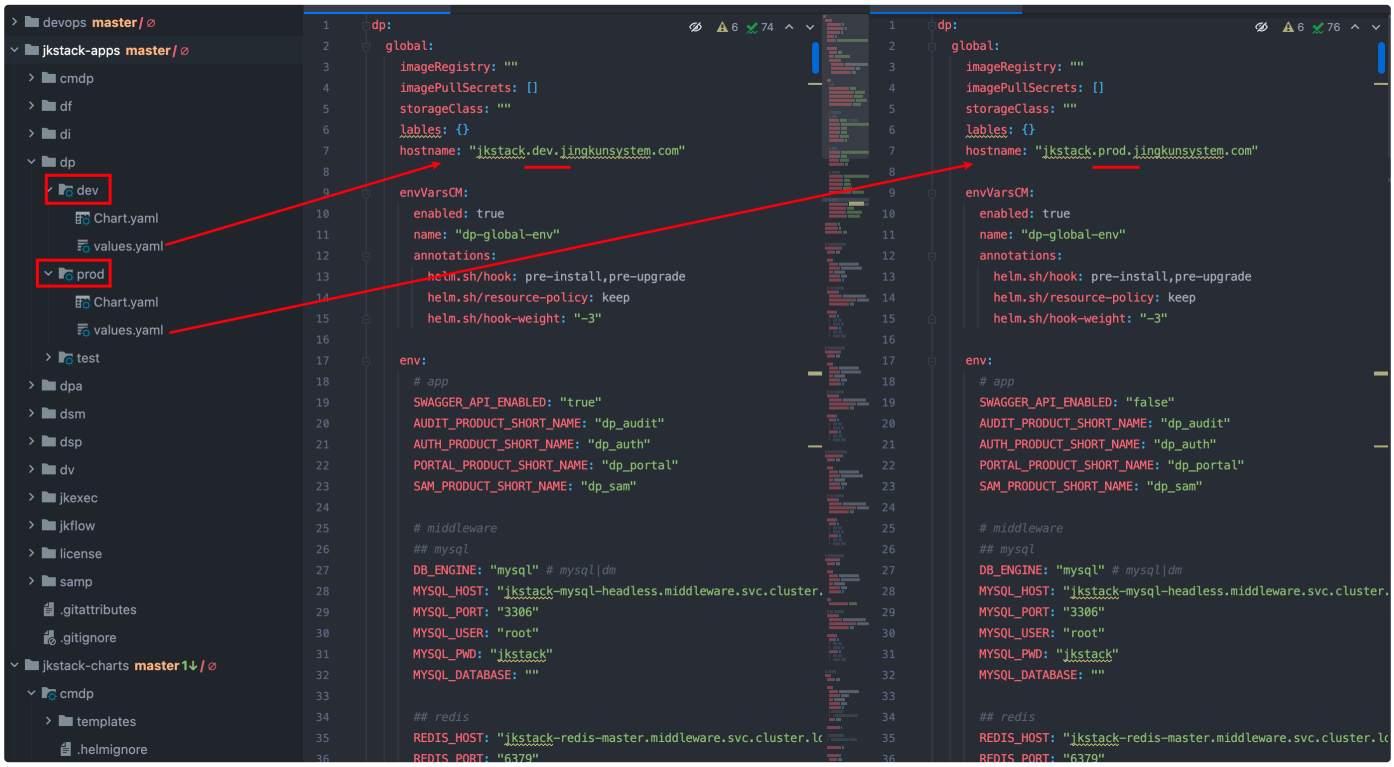


Argo 下产品应用服务编排 **（ARM的部署，在这里作适配）**

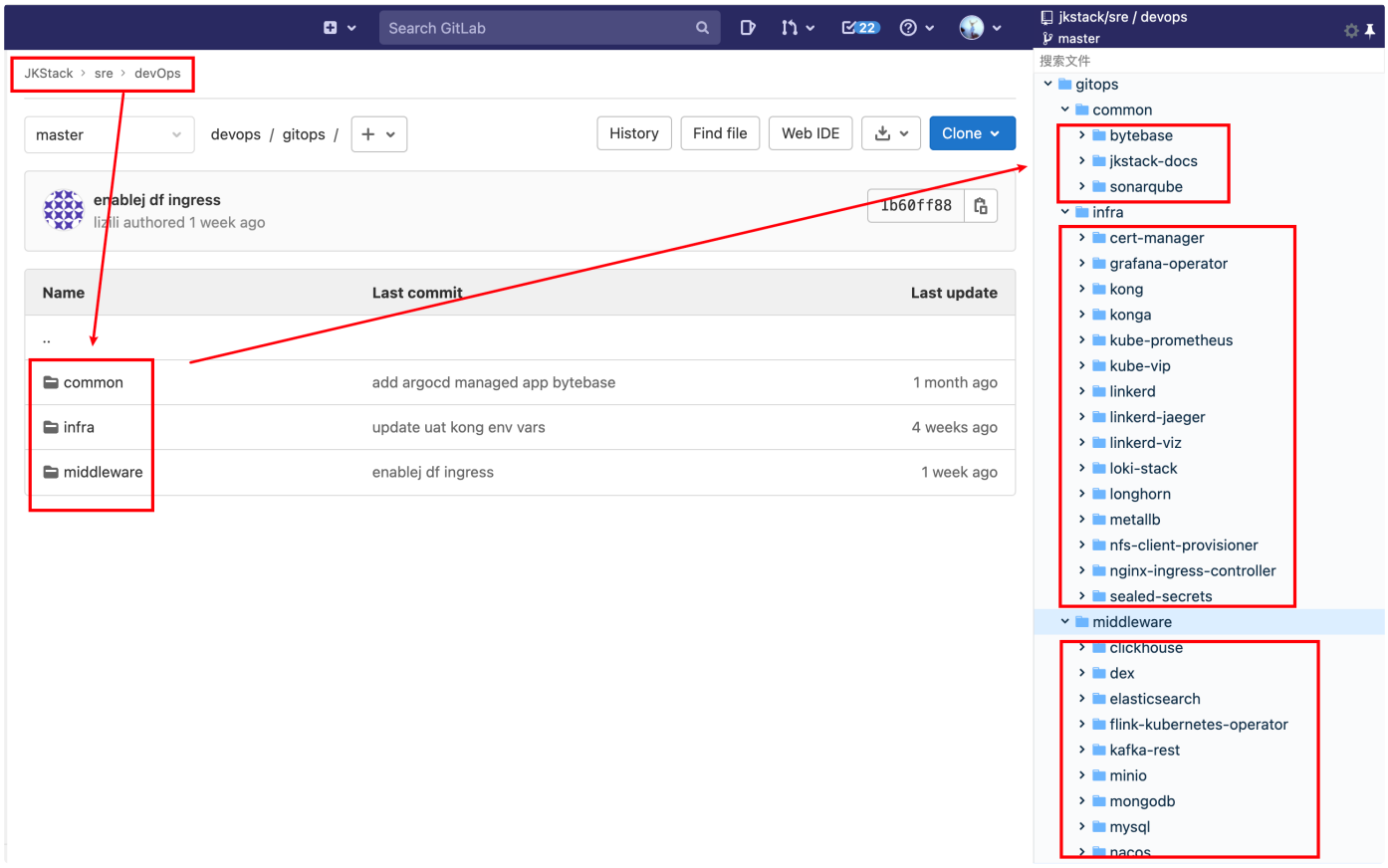
- 命名的区分
- 运行环境指定
  - 可区分不同的环境发布到不同的集群内，达到多环境的隔离
- 编排文件指定
  - 可以指定应用的编排文件，后续在编排中通过申请的通配域名，即可实现多环境的访问切换，如
    - https://jkstack.prod.jingkunsystem.com:29039/
    - https://jkstack.test.jingkunsystem.com:29040/



编排文件对应的就是ArgoCD声明的仓库路径，从而实现了域名和环境的区分，产品的发布只需要修改这里即可，这样后续在pipeline中就可以实现自动化



gitops

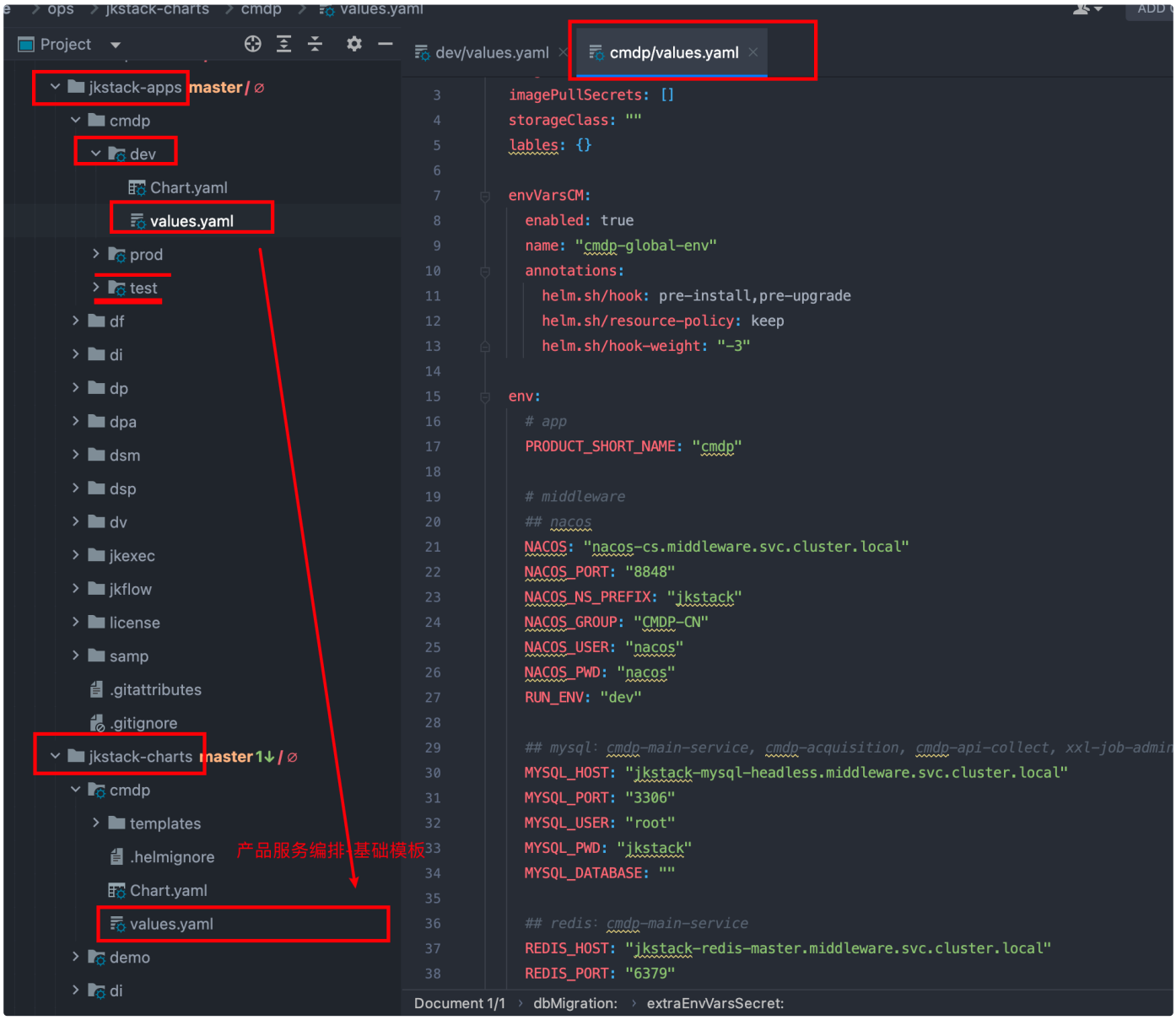


ARM运行环境，主要是在这里作适配

1. common： 公共服务管理，比如SQL审计平台 bytebase，研发文档服务docs 等
2. infra： 基础设施服务，和kubernetes集群相关，如SLB（metallb），存储服务等
3. middleware: 中间件服务，产品依赖中间件的编排均在此维护

4. **jkstack-charts （项目外单独维护）**
    1. 按产线归类，所有的产品运行编排服务均在此仓库进行维护和管理
5. **jkstack-apps （项目外单独维护）**
    1. 产品运行环境归类，基于业务编排模板，进行差异化修改和管理，之后可由**Argo-cd**发布到对应的集群环境中

charts和app介绍



如图

apps中的服务按环境进行区分管理，通过差异化配置供ArgoCD使用，从而发布到不同环境。其核心编排均来自服务编排模板，优势在于

- **避免多环境操作同一个文件，管理混乱**
- **避免多环境同时发布，同文件的冲突**
- **工程文件和基础环境保持了一致**
- **文件改动少，便于自动化**

其他目录

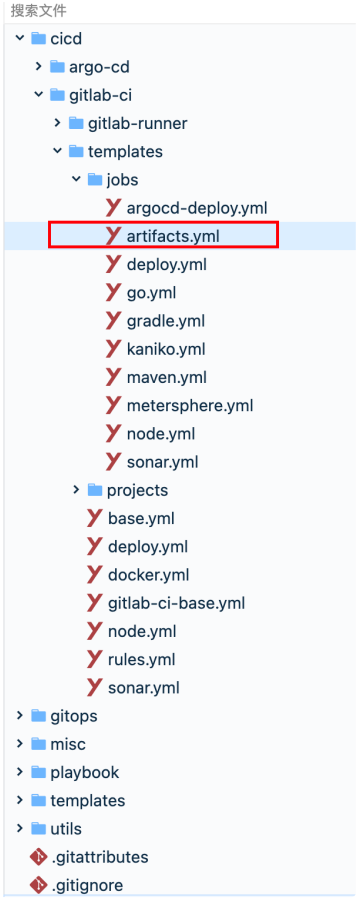
和cicd关系不大，暂不详细介绍

镜像管理

通过harbor管理所有的编排文件和制品镜像 **（Harbor需升级用来支持ARM制品的保存）**

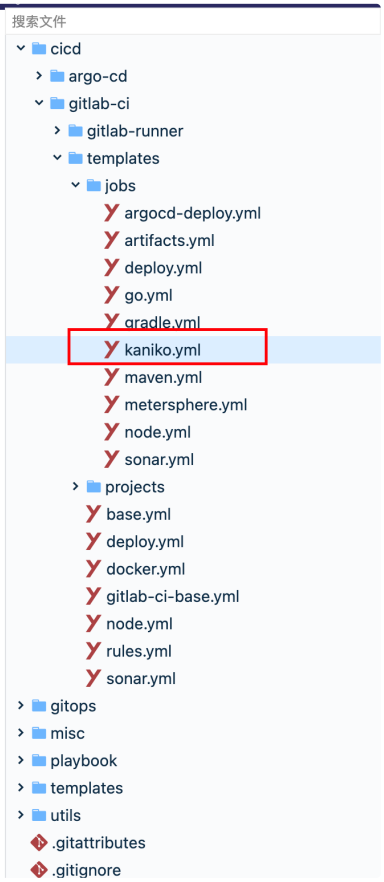
版本上报

```
      echo "IMAGE_TAG: ${IMAGE_TAG}"
      echo "DEPLOY_ENV: ${DEPLOY_ENV}"
      echo "ZIPPED_ARCHIVE_API: ${ZIPPED_ARCHIVE_API}"
      echo "AUTOOPS_URL: ${AUTOOPS_URL}"
    else
      echo 'Fatal: "artifacts:reports:dotenv" variables PRODUCT_VERSION or IMAGE_TAG is null.'
      exit 1
    fi
    if [[ "${DEPLOY_ENV}" = "prod" ]] || [[ "${DEPLOY_ENV}" = "uat" ]];then
      echo "Updating ${DEPLOY_ENV} artifacts info to platform."
    else
      echo "Would not update ${DEPLOY_ENV} artifacts info to platform."
      exit 0
    fi
script:
  - |
    # set -x
    # apk add curl
    AUTOOPS_SCHEME=$(echo "${AUTOOPS_URL}" | awk -F: '{print $1}')
    AUTOOPS_HOST=$(echo "${AUTOOPS_URL}" | sed -n -r 's#(http[s]?:/)?(.+?\.?.+\..{2,3})(:.+)?(/.+)?#\2#p')
    AUTOOPS_PORT=$(echo "${AUTOOPS_URL}" | sed -n -r 's#.+:([0-9]+)(/.+)?#\1#p')
    if [[ -n "${AUTOOPS_PORT}" ]]; then
      AUTOOPS_PORT=${AUTOOPS_PORT}
    elif [[ "${AUTOOPS_SCHEME}" == "https" ]]; then
      AUTOOPS_PORT=443
    else
      AUTOOPS_PORT=80
    fi
    if  nc -z -w 5 ${AUTOOPS_HOST} ${AUTOOPS_PORT}; then
      if [[ -n "${ZIPPED_ARCHIVE_API}" ]]; then
        count=0
        until [[ $count -ge 5 ]]; do
          # ((count++))
          count=$(expr $count + 1)
          echo "Updating ${PRODUCT_NAME}-${SERVICE_NAME} zipped archive package info to autoops platform the
          if curl -X POST \
                  -H  "accept: application/json" \
                  -H  "Content-Type: application/json" \
                  -d "{\"product_name\": \"${PRODUCT_NAME}\", \"product_version\": \"${PRODUCT_VERSION}\", \
              "${AUTOOPS_URL}"
```



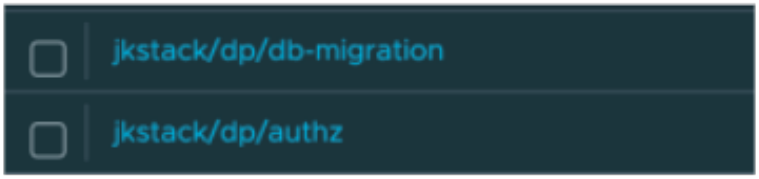
制品上传

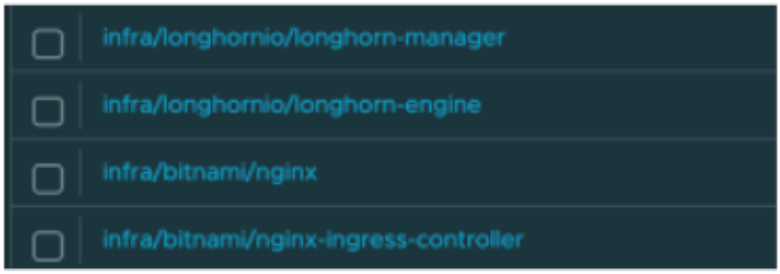
```
    cat ${SERVICE_NAME}-deploy.env
  - echo "Building and shipping image from ${KANIKO_BUILD_CONTEXT_PATH} to ${REGISTRY_HOST}/${REGISTRY_REPO}"
  - mkdir -p /kaniko/.docker
  - echo "{\"auths\":{\"${REGISTRY_HOST}\":{\"auth\":\"$(echo -n ${REGISTRY_USER}:${REGISTRY_PWD} | base64 |
  - DOCKERFILE_PATH=${DOCKERFILE_PATH:-"$KANIKO_BUILD_CONTEXT_PATH/Dockerfile"}
  - |
    # KANIKO_BUILD_ARGS="--build-arg QA_PLATFORM_ADDRESS=${QA_PLATFORM_ADDRESS}
    #                    --build-arg SERVICE_NAME=${SERVICE_NAME}
    #                    --build-arg INCLUDES_PACKAGE=${INCLUDES_PACKAGE}
    #                    --build-arg PROJECT_URL=${CI_PROJECT_URL}
    #                    --build-arg PROJECT_COMMIT_REF_NAME=${CI_COMMIT_REF_NAME}
    #                    --build-arg PROJECT_COMMIT_SHORT_SHA=${CI_COMMIT_SHORT_SHA}"
    if [[ "$CI_MERGE_REQUEST_TARGET_BRANCH_NAME" == "${JKSTACK_PROD_BRANCH_NAME}" ]]; then
      /kaniko/executor --context ${KANIKO_BUILD_CONTEXT_PATH} --dockerfile ${DOCKERFILE_PATH} \
      --skip-tls-verify --skip-tls-verify-pull --insecure --insecure-pull \
      --destination ${REGISTRY_HOST}/${REGISTRY_REPO}/$SERVICE_NAME:${TAG_LATEST} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG_BRANCH}  \
      --destination $REGISTRY_HOST/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO}/${SERVICE_NAME}:current-${DEPLOY_ENV:-latest} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO_DEFAULT}/${PRODUCT_NAME}-${SERVICE_NAME}:${TAG_LATEST} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO_DEFAULT}/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG_BRANCH} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO_DEFAULT}/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG}
    elif [[ "$CI_MERGE_REQUEST_TARGET_BRANCH_NAME" == "${JKSTACK_RELEASE_BRANCH_NAME}" ]]; then
      /kaniko/executor --context ${KANIKO_BUILD_CONTEXT_PATH} --dockerfile ${DOCKERFILE_PATH} \
      ${KANIKO_BUILD_ARGS} \
      --skip-tls-verify --skip-tls-verify-pull --insecure --insecure-pull \
      --destination ${REGISTRY_HOST}/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG_BRANCH} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO}/${SERVICE_NAME}:current-${DEPLOY_ENV:-latest}
    elif [[ "${DEPLOY_ENV}" == "uat" ]]; then
      /kaniko/executor --context ${KANIKO_BUILD_CONTEXT_PATH} --dockerfile ${DOCKERFILE_PATH} \
      --skip-tls-verify --skip-tls-verify-pull --insecure --insecure-pull \
      --destination ${REGISTRY_HOST}/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG_BRANCH} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO}/${SERVICE_NAME}:current-${DEPLOY_ENV:-latest} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO_DEFAULT}/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG_BRANCH} \
      --destination $REGISTRY_HOST/${REGISTRY_REPO_DEFAULT}/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG}
    else
      /kaniko/executor --context ${KANIKO_BUILD_CONTEXT_PATH} --dockerfile ${DOCKERFILE_PATH} \
      --skip-tls-verify --skip-tls-verify-pull --insecure --insecure-pull \
```

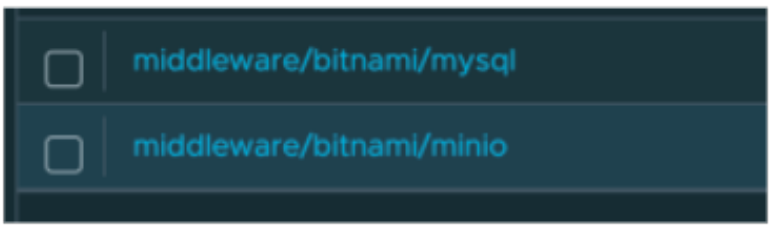


规则：

1. jkstack/<产品>/服务名:版本　　**(jkstack-charts也在此项目中)**
2. middleware/[registry]/服务名:版本
3. infra/[分类]/服务名:版本



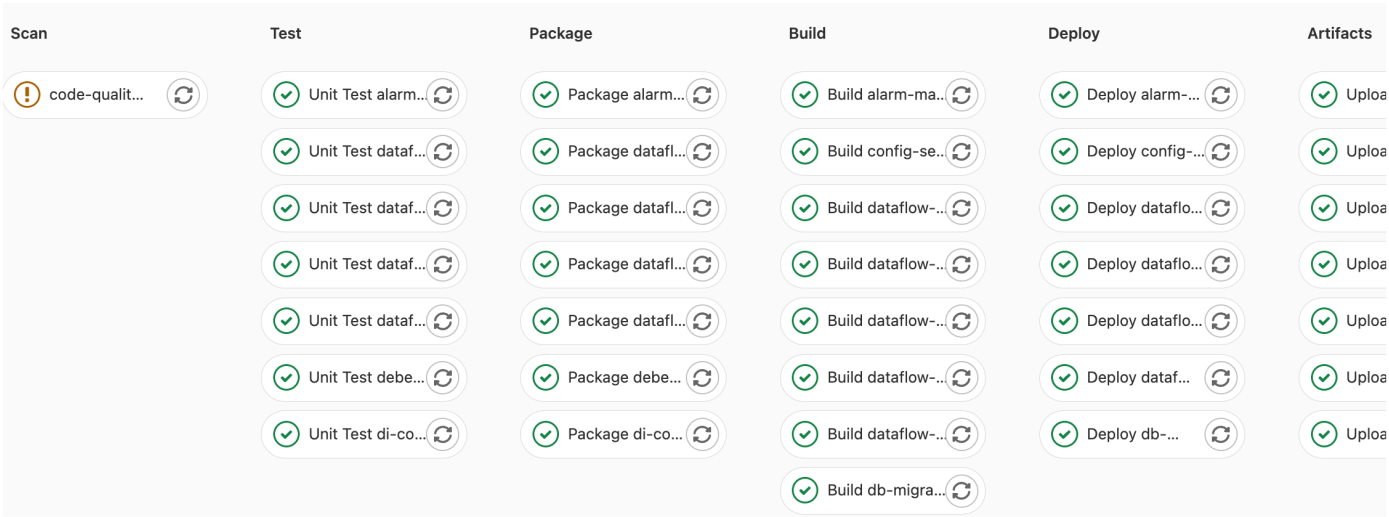


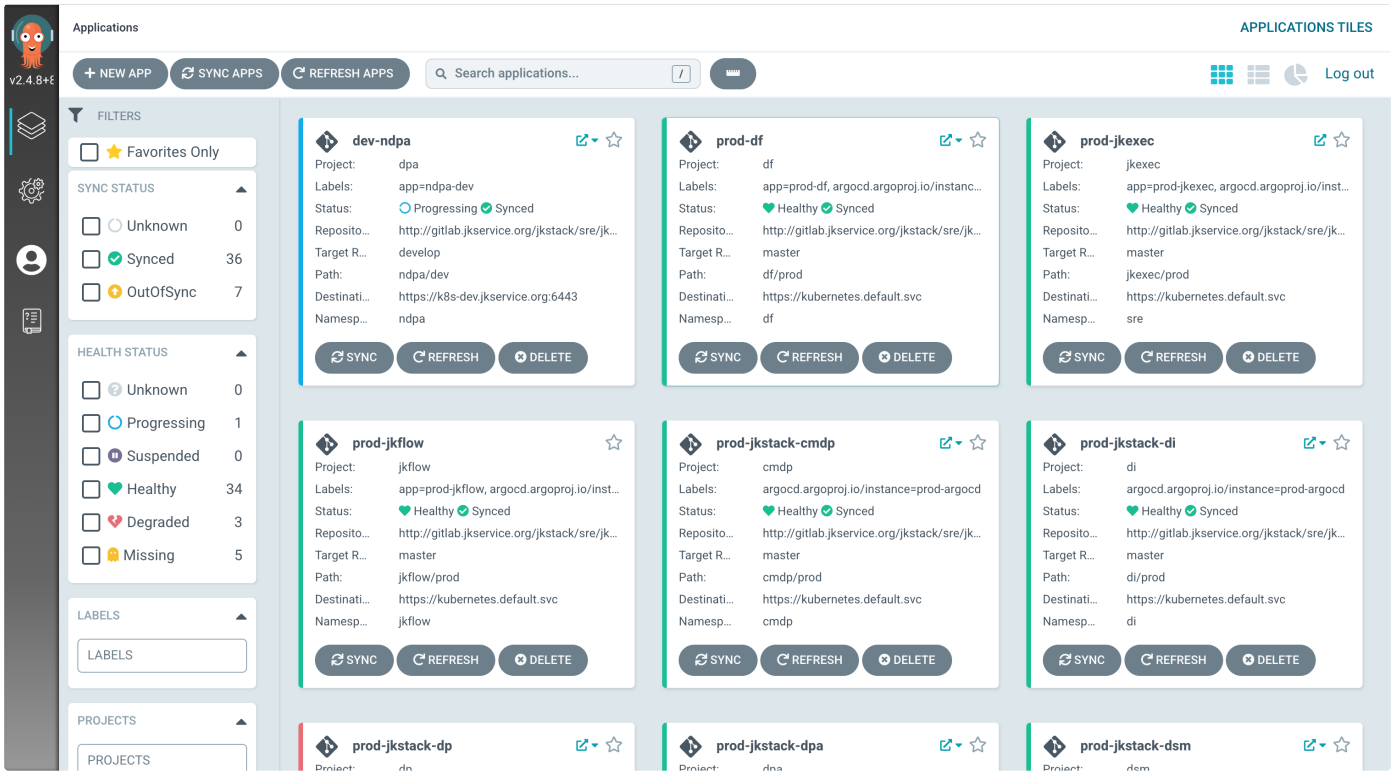


流水线

具备以上条件后后，通过git pipeline 即可将产品的CICD串联起来，实现了多环境的发版控制

Status	Pipeline ID	Triggerer	Commit	Stages	Duration
passed	#45077 latest		feat/102897… 1f04080f Merge branch 'develop' i...		00:01:56 1 week ago
passed	#45075 latest detached		1028 56295a42 Feat/1028978 xc 租房信...		00:02:35 1 week ago
passed	#45057 latest detached		1036 19cd62df update：租房信息上报使...		00:01:52 1 week ago
passed	#45055 detached		1036 17c0d3f9 Merge branch 'develop' i...		00:01:59 1 week ago

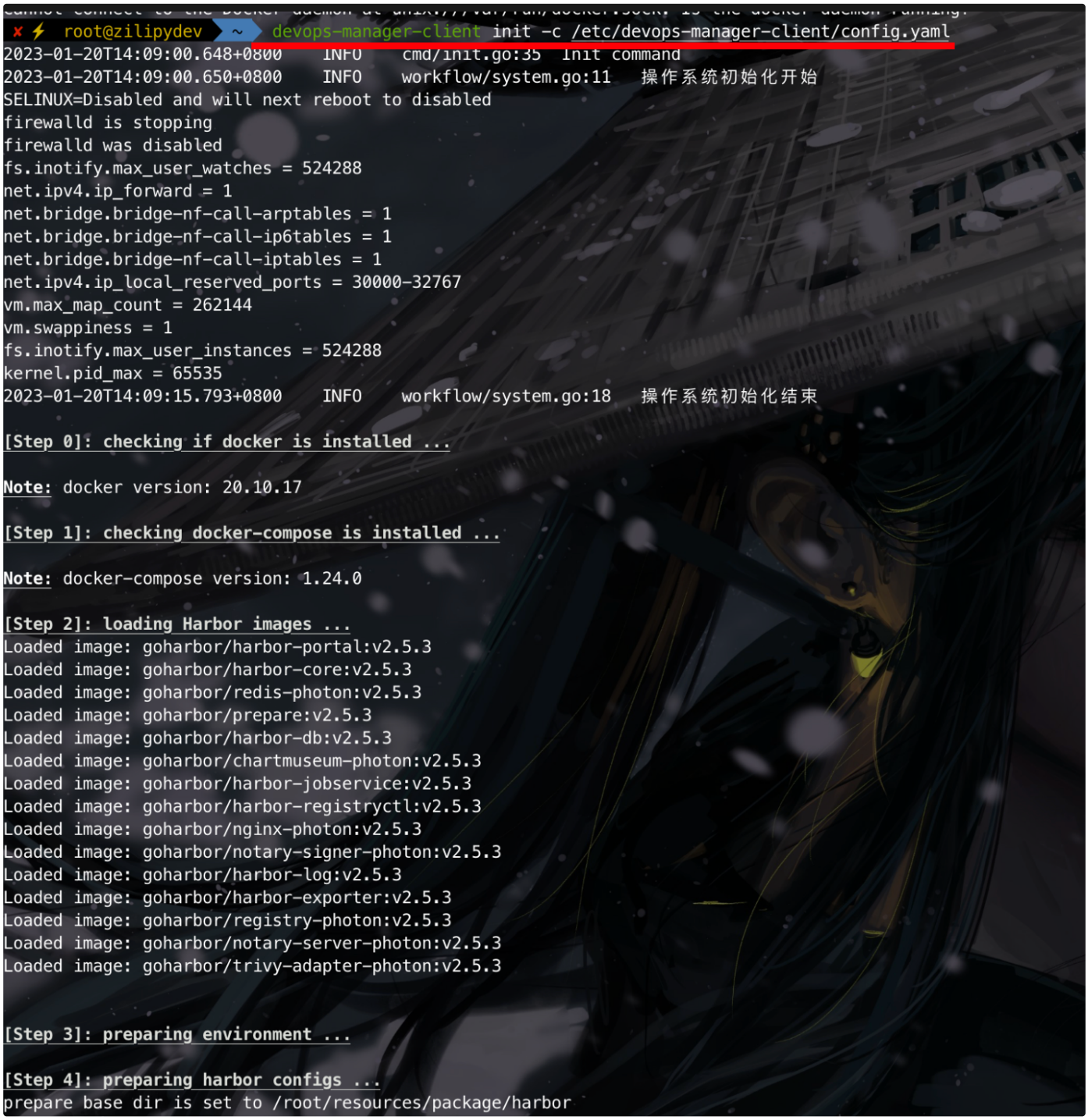




自动化工具

略 （ 需做ARM适配）

1. 版本管理平台
2. 部署客户端（可将产品版本依赖及镜像一键离线、导入导出，并拉起集群环境）


root@zilipydev ~ devops-manager-client init -c /etc/devops-manager-client/config.yaml
2023-01-20T14:09:00.648+0800 INFO cmd/init.go:35 init command
2023-01-20T14:09:00.650+0800 INFO workflow/system.go:11 操作系统初始化开始
SELINUX=Disabled and will next reboot to disabled
firewalld is stopping
firewalld was disabled
fs.inotify.max_user_watches = 524288
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-arptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_local_reserved_ports = 30000-32767
vm.max_map_count = 262144
vm.swappiness = 1
fs.inotify.max_user_instances = 524288
kernel.pid_max = 65535
2023-01-20T14:09:15.793+0800 INFO workflow/system.go:18 操作系统初始化结束
[Step 0]: checking if docker is installed ...
Note: docker version: 20.10.17
[Step 1]: checking docker-compose is installed ...
Note: docker-compose version: 1.24.0
[Step 2]: loading Harbor images ...
Loaded image: goharbor/harbor-portal:v2.5.3
Loaded image: goharbor/harbor-core:v2.5.3
Loaded image: goharbor/redis-photon:v2.5.3
Loaded image: goharbor/prepare:v2.5.3
Loaded image: goharbor/harbor-db:v2.5.3
Loaded image: goharbor/chartmuseum-photon:v2.5.3
Loaded image: goharbor/harbor-jobservice:v2.5.3
Loaded image: goharbor/harbor-registryctl:v2.5.3
Loaded image: goharbor/nginx-photon:v2.5.3
Loaded image: goharbor/notary-signer-photon:v2.5.3
Loaded image: goharbor/harbor-log:v2.5.3
Loaded image: goharbor/harbor-exporter:v2.5.3
Loaded image: goharbor/registry-photon:v2.5.3
Loaded image: goharbor/notary-server-photon:v2.5.3
Loaded image: goharbor/trivy-adapter-photon:v2.5.3
[Step 3]: preparing environment ...
[Step 4]: preparing harbor configs ...
prepare base dir is set to /root/resources/package/harbor

3. jke